

AI Collaborative Communication Platform

Application de communication collaborative en temps réel (type WhatsApp/Discord/Teams) avec assistant IA intégré (résumé, traduction, correction, réponses intelligentes) et authentification par reconnaissance faciale.

Stack technique

Partie	Technologies
Frontend	React, TypeScript, Tailwind CSS, Axios, React Router, Socket.IO Client
Backend	NestJS, TypeScript, Prisma ORM
Base de données	PostgreSQL
Temps réel	WebSocket (Socket.IO)
Authentification	JWT, Refresh Token, BCrypt
IA générative	Ollama (Qwen 2.5)
Reconnaissance faciale	Python, FastAPI, DeepFace, OpenCV
Stockage fichiers	Multer + Cloudinary
Déploiement	Docker, Docker Compose

Prérequis à installer (une seule fois par machine)

Avant de cloner le projet, installe ces logiciels sur ta machine :

1. **Node.js** (LTS) → <https://nodejs.org>

```
bash
node -v
npm -v
```

2. **PostgreSQL** (v16 ou 17) → <https://www.postgresql.org/download/>
 - Note bien le mot de passe du user `postgres` pendant l'installation
 - Vérifie que `psql` fonctionne dans un terminal :

```
bash
```

```
psql --version
```

- Si `psql` n'est pas reconnu, ajoute son dossier `bin` (ex: `C:\Program Files\PostgreSQL\17\bin`) à la variable d'environnement **Path** de Windows, puis redémarre le terminal.
3. **Git** → <https://git-scm.com/downloads>
 4. **Ollama** → <https://ollama.com/download>

```
bash
```

```
ollama pull qwen2.5:3b
```

5. **Python 3.10 ou 3.11** (pour le microservice de reconnaissance faciale) → <https://www.python.org/downloads/>

Installation du projet

1. Cloner le repo

```
bash
```

```
git clone https://github.com/azizjouini724/ai-chat-platform.git  
cd ai-chat-platform
```

2. Créer la base de données PostgreSQL

Connecte-toi à PostgreSQL :

```
bash
```

```
psql -U postgres
```

Puis crée la base :

```
sql
```

```
CREATE DATABASE ai_chat_platform;  
\q
```

3. Configurer le backend

```
bash

cd backend
npm install
```

Copie le fichier d'exemple d'environnement :

```
bash

cp .env.example .env
```

Ouvre `.env` et remplace le mot de passe par le tien :

```
DATABASE_URL="postgresql://postgres:TON_MOT_DE_PASSE@localhost:5432/ai_chat_pla
```

Applique les migrations Prisma (crée les tables) :

```
bash

npx prisma migrate dev
npx prisma generate
```

Lance le serveur backend :

```
bash

npm run start:dev
```

Le backend tourne sur <http://localhost:3000>

4. Configurer le frontend

Dans un nouveau terminal :

```
bash

cd frontend
npm install
npm run dev
```

Le frontend tourne sur <http://localhost:5173>

5. Vérifier que tout fonctionne

- Backend : va sur <http://localhost:3000/users> → tu dois voir `[]`

- Frontend : va sur `http://localhost:5173` → tu dois voir la page React

Commandes utiles

Commande	Description
<code>npx prisma studio</code>	Interface visuelle pour voir les tables de la base
<code>npx prisma migrate dev --name nom</code>	Créer une nouvelle migration après modification du schéma
<code>ollama list</code>	Voir les modèles Ollama installés
<code>ollama run qwen2.5:3b</code>	Tester le modèle IA en ligne de commande

Structure du projet

```
ai-chat-platform/  
├── backend/           # API NestJS + Prisma  
│   ├── prisma/       # Schéma et migrations de la base de données  
│   └── src/           # Code source du backend  
├── frontend/         # Application React  
│   └── src/           # Code source du frontend  
└── README.md
```

Notes importantes

- **Ne jamais commit le fichier `.env`** (il contient des mots de passe) — utilise `.env.example` comme modèle
- **Ne jamais commit `node_modules/`** — chaque personne le régénère avec `npm install`
- Chaque développeur doit avoir sa propre base PostgreSQL locale avec les mêmes migrations appliquées